Модуль 2
Модули и npm

CommonJS • ES Modules • require • import/export • package.json • npm • npx • Зависимости

2.1 Зачем нужны модули

Без модулей весь код приложения находился бы в одном файле. Для маленького скрипта это допустимо, но реальные проекты содержат тысячи строк кода. Модули позволяют разбить код на логические части: один файл отвечает за работу с базой данных, другой — за обработку HTTP-запросов, третий — за отправку email.
Модульная система решает несколько задач:

- Организация кода — каждый файл отвечает за одну задачу (принцип единственной ответственности)
- Переиспользование — написанный модуль можно использовать в разных проектах
- Инкапсуляция — модуль экспортирует только то, что нужно другим, скрывая внутреннюю реализацию
- Управление зависимостями — чётко видно, какой модуль от чего зависит

В Node.js существуют две системы модулей: CommonJS (историческая, используется по умолчанию) и ES Modules (современный стандарт JavaScript). Рассмотрим обе.

2.2 CommonJS модули

CommonJS — это система модулей, которая была в Node.js с самого начала. Она использует функцию require() для импорта и объект module.exports для экспорта. Если вы не указали "type": "module" в package.json, Node.js по умолчанию использует CommonJS.

Экспорт нескольких функций из модуля. Создайте файл math.js

```
// math.js
function add(a, b) {
    return a + b;
}

function multiply(a, b) {
    return a * b;
}

// Экспортируем объект с функциями
module.exports = { add, multiply };
```

Импорт и использование в другом файле. Создайте файл app.js:

```
// app.js
const { add, multiply } = require('./math');

console.log(add(2, 3));       // 5
console.log(multiply(4, 5));  // 20
```

Выполните

```
node app.js
```

Обратите внимание: путь начинается с ./ — это означает «файл в текущей директории». Без ./ Node.js будет искать модуль в папке node_modules или среди встроенных модулей. Расширение .js можно не указывать.
Если модуль экспортирует одно значение (класс, функцию), удобнее присвоить его напрямую. Создайте еще один файл logger.js:

```
// logger.js
class Logger {
    log(msg) { console.log(`[LOG] ${msg}`); }
    error(msg) { console.error(`[ERR] ${msg}`); }
}

module.exports = Logger;  // Экспортируем сам класс, а не объект
```

Измените файл app.js так

```
const { add, multiply } = require('./math');

console.log(add(2, 3));       // 5
console.log(multiply(4, 5));

const Logger = require('./logger');
const log = new Logger();
log.log('Сервер запущен');
```

и запустите для вывода результата

Важная особенность CommonJS: require() работает синхронно. Файл модуля читается и выполняется в момент вызова require(). Результат кешируется — при повторном require() того же файла код не выполняется заново, а возвращается закешированный результат.

2.3 ES Modules (ESM) – Немного теории, без нее никак!

ES Modules (ESM) — это стандарт модульной системы, принятый в спецификации ECMAScript (ES6, 2015). Он использует ключевые слова import и export для организации кода в отдельные, переиспользуемые модули.

Зачем нужны модули?
Без модулей весь код находился бы в одном файле или в глобальной области видимости, что приводит к проблемам:

- Конфликты имён — две переменные с одинаковым именем перезаписывают друг друга
- Сложность поддержки — в файле на 5000 строк невозможно быстро найти нужную функцию
- Невозможность переиспользования — один и тот же код приходится копировать между проектами

Модули решают все эти проблемы: каждый файл — изолированная единица со своей областью видимости, которая явно объявляет, что она экспортирует (отдаёт наружу) и что импортирует (берёт из других модулей).

Отличие ESM от CommonJS

Характеристика	CommonJS (require)	ESM (import/export)
Синтаксис	const x = require('./x')	import x from './x'
Загрузка	Синхронная (блокирующая)	Асинхронная (неблокирующая)
Время разбора	В момент выполнения (runtime)	До выполнения, на этапе парсинга (статический анализ)
Где работает	Только Node.js	Браузеры + Node.js
Tree-shaking	Не поддерживается	Поддерживается (бандлеры могут удалить неиспользуемый код)
Top-level await	Не поддерживается	Поддерживается

this на верхнем уровне	Ссылается на module.exports	undefined
__dirname / __filename	Доступны глобально	Нет, используйте import.meta.url

Tree-shaking — это важное преимущество ESM. Поскольку импорты анализируются статически (до выполнения кода), сборщики (Webpack, Rollup, esbuild) могут определить, какие функции реально используются, и удалить неиспользуемый код из финальной сборки. С CommonJS это невозможно, потому что require() может вызываться динамически внутри if-блоков.

Как включить ESM в Node.js

Чтобы использовать ESM в Node.js, нужно сделать одно из двух:

Способ 1: Указать "type": "module" в package.json

```
{
    "name": "my-project",
    "version": "1.0.0",
    "type": "module"   // <-- эта строка включает ESM для ВСЕХ .js
файлов
}
```

После этого все файлы .js в проекте будут интерпретироваться как ES-модули. Если вам нужен отдельный файл в формате CommonJS, дайте ему расширение .cjs.

Способ 2: Использовать расширение .mjs

Файлы с расширением .mjs всегда обрабатываются как ES-модули, независимо от настроек package.json. Это удобно, когда нужно использовать ESM в проекте, который в целом работает на CommonJS.

```
my-project/
├── package.json          (без "type": "module")
├── app.js                <-- CommonJS (require/module.exports)
├── utils.mjs             <-- ESM (import/export)
└── helpers.cjs           <-- Явно CommonJS
```

Рекомендация: В новых проектах лучше использовать Способ 1 ("type": "module") — это современный подход, и большинство новых библиотек Node.js уже переходят на ESM.

Функции из модулей могут быть экспортированы, а также импортированы в этот модуль из других модулей. Существует несколько видов экспорта и импорта.

Именованный экспорт (Named Export)
Именованный экспорт позволяет экспортировать несколько значений из одного модуля. Каждое значение экспортируется по своему имени — и импортировать его нужно точно по этому имени.

```
// math.js (с "type": "module" в package.json)

// Можно ставить export перед каждым объявлением:
export function add(a, b) {
    return a + b;
}

export function multiply(a, b) {
    return a * b;
}

export const PI = 3.14159;

// Приватная функция — НЕ экспортирована, недоступна снаружи
function _internalHelper() {
    return 42;
}
```

add, multiply и PI — экспортированы, их можно импортировать в другом файле.
_internalHelper — не экспортирована, она существует только внутри math.js и недоступна извне.
Это ключевое свойство модулей: вы контролируете, что видно снаружи, а что скрыто.

Альтернативный синтаксис экспорта — экспорт списком в конце файла:

```
// math.js

function add(a, b) {
    return a + b;
}

function multiply(a, b) {
    return a * b;
}

const PI = 3.14159;

// Экспортируем все разом в конце файла
export { add, multiply, PI };
```

Оба варианта полностью эквивалентны. Первый (export перед объявлением) удобнее, когда функций много. Второй удобнее, когда хотите видеть весь «публичный API» модуля в одном месте.

Экспорт с переименованием (alias):

```
// math.js
function add(a, b) { return a + b; }

export { add as sum };  // снаружи будет доступна как sum, а не add
```

Дефолтный экспорт (Default Export)

Дефолтный экспорт — это одно главное значение из модуля. Каждый модуль может иметь только один дефолтный экспорт.

```
// Calculator.js
export default class Calculator {
    add(a, b) { return a + b; }
    sub(a, b) { return a - b; }
    mul(a, b) { return a * b; }
    div(a, b) {
        if (b === 0) throw new Error('Деление на ноль!');
        return a / b;
    }
}
```

Когда использовать дефолтный экспорт:
Когда модуль представляет одну сущность — один класс, одну функцию, один объект. Например: класс Calculator, функция createServer, объект конфигурации.

Когда использовать именованный экспорт:
Когда модуль содержит набор утилит — несколько функций, констант. Например: { add, multiply, PI } из математического модуля.

Важно: можно комбинировать оба типа экспорта в одном файле, но это не рекомендуется — это запутывает пользователей модуля.

Именованный импорт — фигурные скобки обязательны:

```
// app.js
import { add, multiply, PI } from './math.js';

console.log(add(2, 3));       // 5
console.log(multiply(4, 5));   // 20
console.log(PI);               // 3.14159
```

Обратите внимание:
Имена в { } должны точно совпадать с именами экспорта
Расширение .js — обязательно в ESM Node.js (в отличие от CommonJS, где можно опустить)

Дефолтный импорт — без фигурных скобок:

```
import Calculator from './Calculator.js';

const calc = new Calculator();
console.log(calc.add(10, 20));  // 30
console.log(calc.div(10, 3));   // 3.333...
```

При дефолтном импорте вы можете назвать переменную как угодно:

```
import MyCalc from './Calculator.js';      // Работает
import Калькулятор from './Calculator.js'; // Тоже работает
```

Импорт с переименованием (alias):

```
import { add as сложить, multiply as умножить } from './math.js';

console.log(сложить(2, 3));    // 5
console.log(умножить(4, 5));   // 20
```

Это полезно, когда имена из разных модулей конфликтуют или когда хотите дать более понятное имя в контексте текущего файла.

Импорт всего модуля как объекта (namespace import):

```
import * as math from './math.js';

console.log(math.add(2, 3));       // 5
console.log(math.multiply(4, 5));  // 20
console.log(math.PI);              // 3.14159
```

Удобно, когда модуль экспортирует много функций и вы хотите явно показать, откуда берётся каждая.

Импорт встроенных модулей Node.js:

```
import { readFile } from 'fs/promises'; // Промис-версия fs
import { join, resolve } from 'path';   // Работа с путями
import { createServer } from 'http';    // HTTP-сервер
// Также можно использовать префикс node: (рекомендуется)
import { readFile } from 'node:fs/promises';
```

Префикс node: рекомендуется в современном коде. Он явно указывает, что это встроенный модуль Node.js, а не npm-пакет с таким же именем.

Комбинированный импорт (дефолтный + именованный):

Если модуль экспортирует и default, и именованные значения, то импортировать можно таким образом:

```
import Calculator, { helperFunction } from './Calculator.js';
```

Динамический импорт

В отличие от статического import, динамический import() загружает модуль во время выполнения (runtime). Возвращает Promise.

```
async function loadModule(type) {
    if (type === 'math') {
        const math = await import('./math.js');
        console.log(math.add(2, 3));     // 5
    } else {
        const utils = await import('./utils.js');
        utils.doSomething();
    }
}
```

Когда использовать:

- Ленивая загрузка (lazy loading) — загрузить тяжёлый модуль только когда он действительно нужен
- Условная загрузка — выбрать модуль в зависимости от настроек или окружения
- Загрузка CommonJS-модулей из ESM-кода
- Top-level await

В ESM можно использовать await на верхнем уровне файла, без обёртки в async function:

```
// config.js (ESM)
import { readFile } from 'node:fs/promises';

// await прямо на верхнем уровне — работает только в ESM!
const raw = await readFile('./config.json', 'utf-8');
export const config = JSON.parse(raw);
// app.js
import { config } from './config.js';
// config уже загружен и распарсен к этому моменту
console.log(config.port);  // 3000
```

В CommonJS использовать await на верхнем уровне невозможно — там пришлось бы оборачивать в async function и вызывать её.

Частые ошибки и подводные камни

1. Забыли расширение файла

```
// ❌ ОШИБКА в ESM Node.js
import { add } from './math';

// ✅ Правильно — расширение обязательно
import { add } from './math.js';
```

2. Попытка использовать require() в ESM

```
// ❌ ОШИБКА — require не определён в ESM
const fs = require('fs');

// ✅ Правильно
import fs from 'node:fs';
```

3. В ESM нет глобальных __dirname и __filename. Вместо них:

```
import { fileURLToPath } from 'node:url';
import { dirname } from 'node:path';

const __filename = fileURLToPath(import.meta.url);
const __dirname = dirname(__filename);

console.log(__filename);  // /Users/alex/project/app.js
console.log(__dirname);   // /Users/alex/project
```

4. Циклические зависимости

ESM обрабатывает циклические зависимости лучше CommonJS, но их всё равно следует избегать:

```
// a.js
import { b } from './b.js';
export const a = 'A';

// b.js
import { a } from './a.js';  // a может быть undefined на момент
импорта!
export const b = 'B';
```

Итоговая сводка

Операция	Синтаксис
Именованный экспорт	export function add() {} или export { add, PI }
Дефолтный экспорт	export default class Calculator {}
Именованный импорт	import { add, PI } from './math.js'
Дефолтный импорт	import Calculator from './Calculator.js'
Импорт всего	import * as math from './math.js'
Импорт с alias	import { add as sum } from './math.js'
Динамический импорт	const mod = await import('./math.js')
Встроенный модуль	import { readFile } from 'node:fs/promises'

Для новых проектов рекомендуется использовать ES Modules. В этом курсе мы будем использовать ESM.

Самая сложная часть теории закончилась. Дальше будет проще!

2.4 Встроенные модули Node.js

Node.js поставляется с набором встроенных модулей, которые не нужно устанавливать. Они покрывают основные задачи серверной разработки:

- fs — работа с файловой системой (чтение, запись, удаление файлов)
- path — безопасная работа с путями файлов (кроссплатформенно)
- http / https — создание HTTP-серверов и выполнение запросов
- os — информация об операционной системе (CPU, память, платформа)
- crypto — криптографические функции (хеширование, шифрование)
- events — система событий (EventEmitter)
- url — парсинг и форматирование URL
- stream — потоковая обработка данных
- util — вспомогательные утилиты (promisify, inspect)
- child_process — запуск внешних процессов и команд

В следующих частях этого курса мы рассмотрим некоторые из встроенных модулей Node.js

Далее попробуем создать свой модуль и поработать в нем, чтобы все стало более понятно. Создайте новую папку и в ней новый файл package.json с содержимым

```
{
    "name": "my-test-module",
    "version": "1.0.0",
    "type": "module"
}
```

Теперь создадим новый файл mypath.js с содержимым

```
import path from 'node:path';

// path.join() — соединяет части пути
console.log(path.join('/users', 'alex', 'docs', 'file.txt'));
// /users/alex/docs/file.txt

// path.resolve() — возвращает абсолютный путь
console.log(path.resolve('src', 'index.js'));
// /home/alex/project/src/index.js

// path.extname() — расширение файла
console.log(path.extname('photo.jpg'));   // .jpg

// path.basename() — имя файла
console.log(path.basename('/a/b/c.txt')); // c.txt

// path.dirname() — директория файла
console.log(path.dirname('/a/b/c.txt'));  // /a/b
```

Здесь мы импортируем дефолтным импортом встроенный в Node.js модуль path. Модуль path имеет методы (join(), resolve(), extname(), basename(), dirname() и другие), которые позволяют работать с путями в файловой системе. Его используют для объединения, нормализации, извлечения частей пути и т.д.

Модуль path особенно важен, потому что разделители путей различаются на разных ОС: Windows использует обратный слэш (\), а macOS и Linux — прямой (/). Функции path автоматически используют правильный разделитель.

Запустим файл node mypath.js и проверим вывод.

Далее попробуем еще один метод, который предоставляет информацию об операционной системе. Создадим новый файл myos.js с содержимым

```
import os from 'os';

console.log(os.platform());    // darwin, linux, win32
console.log(os.arch());        // x64, arm64
console.log(os.cpus().length); // Количество ядер CPU
console.log(os.totalmem());    // Общая оперативная память (байт)
console.log(os.freemem());     // Свободная память
console.log(os.homedir());     // Домашняя директория пользователя
console.log(os.hostname());    // Имя компьютера
```

Запустим файл node myos.js и проверим вывод.

Для Node.js уже созданы миллионы различных модулей — это библиотеки и пакеты для решения практически любых задач:

- работа с файлами и файловой системой
- отправка электронной почты
- подключение к базам данных
- создание веб-серверов
- работа с API
- аутентификация и безопасность
- логирование
- тестирование
- и многое другое

Чтобы разработчикам было удобно делиться этими модулями друг с другом, устанавливать их в проекты и управлять версиями, был создан централизованный реестр пакетов — npm registry.

2.5 npm и package.json

npm (Node Package Manager) — менеджер пакетов, который устанавливается вместе с Node.js. Он решает две задачи: управление зависимостями вашего проекта и доступ к реестру из более чем 2 миллионов пакетов. Для поиска необходимых модулей (зависимостей) обычно используют сайт https://www.npmjs.com

Открываете сайт, в строке поиска вбиваете ключевое слово, например email, жмакаете Search и сайт выдает все модули, найденные по этому ключевому слову. При выборе модуля следует обращать внимание на:

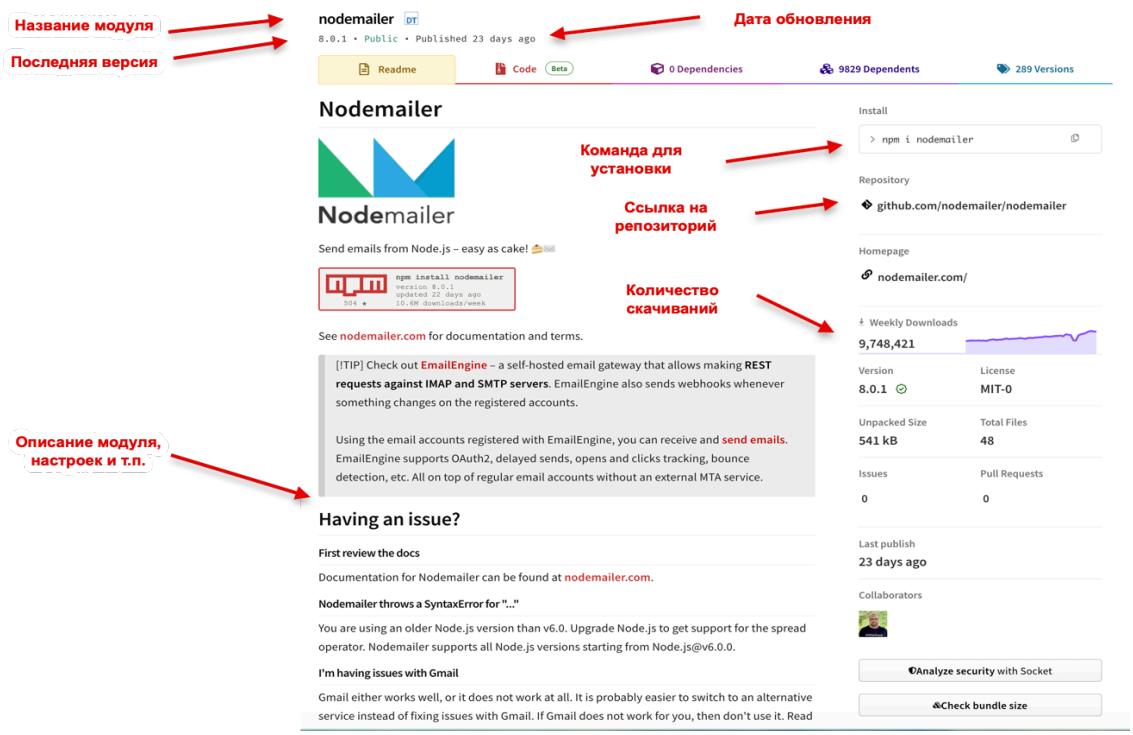
- Количество скачиваний в неделю — показатель популярности
- Количество звезд на GitHub
- Дата последнего обновления
- Наличие документации

Например, для отправки почты часто используют пакет nodemailer (https://www.npmjs.com/package/nodemailer).

Открыв страницу пакета, вы увидите:

- описание
- примеры использования
- команду для установки
- список версий

Сайт npm реестра на английском языке, русской версии не существует. Далее небольшая шпаргалка для понимания, где, что искать:



Теперь давайте создадим новый проект и попробуем скачать какой-нибудь модуль. Самый простой модуль, который подойдет нам для тестирования – это модуль cowsay. Он не требует никакой настройки, не зависит от внешних сервисов и сразу выводит результат в консоль в виде ASCII-коровы 🙂

Создаем новую папку testcowsay и инициализируем файл package.json с помощью npm.

```
mkdir testcowsay
cd testcowsay
npm init -y    # -y автоматически заполняет значения по умолчанию
```

Каждый Node.js-проект всегда начинается с файла package.json — манифеста проекта. Он содержит имя, версию, зависимости и скрипты. Этот файл можно создать вручную, а можно с помощью npm init как мы сделали только что.
Рассмотрим структуру package.json подробно:

```
{
  "name": "testcowsay", // Имя проекта (без пробелов, нижний регистр)
  "version": "1.0.0",   // Версия (semver: major.minor.patch)
  "main": "index.js",   // Точка входа
  "scripts": {          // Скрипты для запуска
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],       // Ключевые слова
  "author": "",         // Автор
  "license": "ISC",     // Лицензия
  "description": ""     // Описание
}
```

Раздел scripts — это именованные команды, которые запускаются через npm run. Скрипты start и test — особые, их можно запускать без run: npm start, npm test.

Основные команды npm:

```
npm install cowsay    # Установить пакет (добавляется в dependencies)
npm install -D cowsay # Установить Dev-зависимость (добавляется в
devDependencies)
npm install           # Установить все зависимости из package.json
npm uninstall cowsay  # Удалить пакет
npm update            # Обновить пакеты до последних совместимых версий
npm list              # Список установленных пакетов
npm list --depth=0    # Только верхнеуровневые пакеты
npm outdated          # Показать устаревшие пакеты
npm run dev           # Запустить скрипт из package.json
```

Разница между dependencies и devDependencies:

- dependencies — для работы приложения (например модули express, mongoose),
- devDependencies — только для разработки (тесты, линтеры, сборщики и т.п.).

При деплое на продакшн devDependencies обычно не устанавливаются (npm install --production).

Наверно у вас возник вопрос, а почему в файле package.json отсутствует параметр type, ведь мы же хотим использовать ESM модули. Давайте изменим файл package.json таким образом:

```
{
  "name": "testcowsay",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": "",
  "type": "module" // <-- Добавим type=module
}
```

Вот теперь все в порядке и наш проект использует ESM модули.

Теперь давайте установим модуль cowsay в проект, выполнив:

```
npm install cowsay
```

И посмотрим содержимое файл package.json еще раз

```
{
  "name": "testcowsay",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": "",
  "type": "module",
```

```
  "dependencies": {
    "cowsay": "^1.6.0" // <-- Вот и наш модуль версии 1.6.0
  }
}
```

Далее создадим новый файл index.js и импортируем модуль cowsay следующим образом:

```
import cowsay from "cowsay";

console.log(
  cowsay.say({
    text: "Hello ESM!",
  })
);
```

Запустим файл, выполнив в консоли

```
node index.js
```

Проверим вывод. Должно быть что-то вроде этого

```
____________
< Hello ESM! >
 ------------
        \   ^__^
         \  (oo)\_______
            (__)\       )\/\
                ||----w |
                ||     ||
```

Это значит, что все сделано правильно.

2.6 Версионирование (Semver)

npm использует семантическое версионирование (Semantic Versioning):

MAJOR.MINOR.PATCH.

- MAJOR (4.x.x → 5.0.0) — несовместимые изменения API
- MINOR (4.18.x → 4.19.0) — новая функциональность, обратно совместимая
- PATCH (4.18.2 → 4.18.3) — исправления багов

Префиксы в package.json:

- ^4.18.2 (каретка) — разрешает обновления MINOR и PATCH (4.18.2 → 4.99.99), но не MAJOR
- ~4.18.2 (тильда) — разрешает только PATCH (4.18.2 → 4.18.99)
- 4.18.2 (без префикса) — точная версия, никаких обновлений

Файл **package-lock.json** фиксирует точные версии всех зависимостей (включая вложенные). Его рекомендуется всегда коммитить в git — он гарантирует, что у всех разработчиков одинаковые версии пакетов.

Практика

Задание 2.1:

- Создайте проект с npm init, добавьте "type": "module"
- Найдите модуль с названием nanoid в реестре и установите его в проект
- Установите модуль chalk (https://www.npmjs.com/package/chalk), затем попробуйте разобраться в описании модуля самостоятельно и выведете цветной текст в консоль
- Изучите package-lock.json — найдите, какие вложенные зависимости установил chalk